Language Models and Agents — Monsoon 2026

Individual Project: Phase 1 Report

Manya Jain

1 Language Selection

1.1 Model H (Higher-resource): Hindi

Hindi was chosen as the higher-resource language because of the sheer volume of public text available for it. Large open-source Indic corpora, complete Wikipedia dumps, and government press releases (PIB) all provide a solid foundation for training a well-resourced monolingual model, satisfying the phase requirement of a non-English Indian language with abundant public text.

1.2 Model L (Lower-resource): Nepali

Nepali was chosen as the lower-resource language. It is one of the languages explicitly permitted by the assignment, and its script and morphology present a meaningfully different tokenization problem from Hindi. Although resources are more limited than for Hindi, a workable corpus was assembled from a large academic dataset (IRIIS) supplemented with targeted news scraping.

2 Dataset Requirements

Both models are built on fully independent monolingual corpora, each with its own train, validation, and test split (90/5/5, seed 42). No documents, sentences, or vocabulary are shared between the two languages.

2.1 Model H: Hindi

The Hindi corpus combines two large downloaded sources with several manually collected ones.

Category	Source	Word Count
Downloaded	Sangraha corpus	194,198,603
Downloaded	Indic corpus	176,448,929
Manual	Hindi Wikipedia	42,801,209
Manual	News scrape (new)	42,383,013
Manual	Subtitles	5,234,233
Manual	News scrape	360,205
Manual	PIB releases	5,885

Table 1: Hindi corpus sources and word counts (raw, pre-cleaning).

The data was cleaned using Unicode NFC normalization, exact-content deduplication, and removal of non-Devanagari (English) characters to keep the corpus script-pure.

Total word count is approximately 461.4 million, which corresponds to roughly 556.1 million tokens after tokenization (fertility 1.2052), exceeding the 500M-token target. Wikipedia, subtitle, and PIB sources to-

gether make up the manually collected portion, giving a manual fraction of about 20% of total words. Total Post-Cleaning Words: 455,866,707 words (Note: The raw source collection was 461,432,077 words. Approximately 5.5M words were removed during exact-content deduplication and cleaning).

Split	Words
Train	410,362,761
Validation	22,804,967
Test	22,698,979

Table 2: Hindi train/validation/test split (90/5/5, seed 42), computed on the post-cleaning corpus of 455,866,707 words.

2.2 Model L: Nepali

The Nepali corpus was built with the manual-collection requirement as the primary constraint, then supplemented with a large downloaded academic dataset.

Category	Source	Word Count
Downloaded	IRIIS Nepali	400,000,000
Manual	News scrape (43 portals)	107,100,815
Manual	Nepali Wikipedia	7,479,341

Table 3: Nepali corpus sources and word counts (raw, pre-cleaning). News data was collected from 43 high-traffic Nepali news portals, respecting robots.txt.

Cleaning steps mirror those used for Hindi: cross-split exact deduplication, whitespace normalization, Unicode NFC normalization, and a Devanagari purity filter.

Total word count is approximately 514.6 million, corresponding to roughly 620.5 million tokens (fertility 1.2059). Manually collected data accounts for about 22.3% of total words (114.5 million), comfortably above the 20% minimum. Both the token target and the manual-collection requirement are met. Total Post-Cleaning Words: 513,019,586 words (Note: The raw source collection was 514,580,156 words. Approximately 1.56M words were removed during exact-content deduplication and cleaning).

Split	Words
Train	502,683,226
Validation	5,194,505
Test	5,141,855

Table 4: Nepali train/validation/test split (90/5/5, seed 42), computed on the post-cleaning corpus.

3 Tokenizer Training

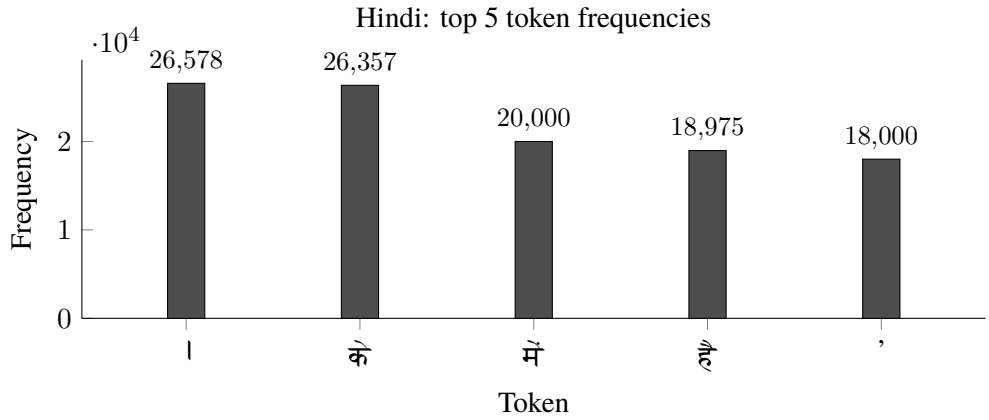
Separate byte-level BPE tokenizers were trained from scratch for each language; no pretrained tokenizer or shared vocabulary was used at any point.

3.1 Model H: Hindi

- Vocabulary size: 32,000

- Average characters per token: 4.22
- Average tokens per word (fertility): 1.2052
- Unknown-token rate: 0.00%

The most frequent tokens are punctuation and common postpositions/verbs (।, के, में, है), which is expected for a well-formed Devanagari tokenizer trained on news and encyclopedic text.



Vocabulary size was chosen to balance tokenization efficiency against coverage. At 32,000 tokens, the Hindi tokenizer achieves a low fertility rate of approximately 1.2052 tokens per word while producing zero unknown tokens on held-out evaluation text, indicating that a larger vocabulary would add overhead without improving coverage.

Example: the sentence "मैं बंदूक नकलनेवल हूँ।" tokenizes into the id sequence `[1458, 8944, 9138, 313, 1016, 2094, 284]`.

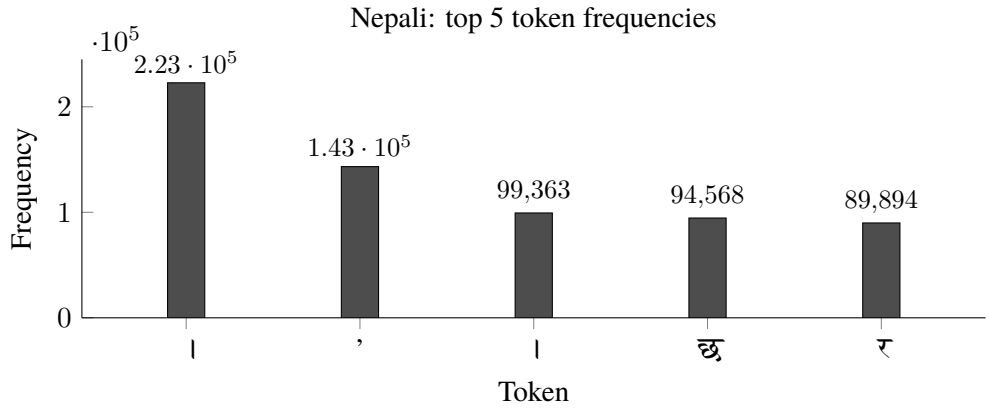
3.2 Model L: Nepali

- Vocabulary size: 37,000
- Average characters per token: 5.38
- Average tokens per word (fertility): 1.2059
- Unknown-token rate: ≈0.00011% (7 tokens)

As with Hindi, the most frequent tokens are punctuation and function words (।, ,, छ, र). If you look closely at the list, the danda (।) appears twice with two different frequencies: once with a leading space before it (222,735 occurrences) and once with no space before it (99,363 occurrences). Tokenizers treat a character with a preceding space as a totally separate entity from the character on its own. This actually highlights a very common real-world data quirk: in the Nepali corpus, people frequently type a space before the full stop, but they also frequently type it directly attached to the end of the last word. Because both variations are so common in the raw text, the tokenizer learned both as distinct, highly frequent tokens.

At 37,000 tokens, the Nepali vocabulary reaches the same fertility rate as Hindi (≈1.2059 tokens per word) while leaving only 7 unknown tokens out of roughly 6.2 million evaluated – a negligible 0.00011% UNK rate. A larger vocabulary was needed relative to Hindi's 32,000 to reach comparable fertility, consistent with Nepali's morphology and script requiring more distinct subword units for equivalent coverage.

Example: the sentence "कश्मीरीकाअर्थ होसकताहै।" tokenizes into the id sequence `[273, 1986, 9232, 334, 1430, 347, 732, 6632]`.



3.3 Tokenizer Architecture Differences

Model H and Model L deliberately use two different BPE variants, chosen from the start of the project as a point of comparison rather than as an artifact of separate implementations:

- **Model H (Hindi) – Byte-level BPE.** Text is first decomposed into raw UTF-8 bytes (the same scheme used by GPT-2 and RoBERTa) before merges are learned. Because every possible byte is representable, this variant can encode any input, including rare symbols, mixed script, or noise, with zero out-of-vocabulary tokens (observed UNK rate: 0.00%). This suits Hindi's larger, more heterogeneous corpus (news, subtitles, Wikipedia, PIB releases), which is more likely to contain incidental non-Devanagari or irregular characters despite the script-purity filtering.
- **Model L (Nepali) – Unicode character-level BPE.** Merges are learned directly over Devanagari Unicode codepoints rather than bytes. This keeps the vocabulary file human-readable in native script and ties each token directly to a valid character, at the cost of a small non-zero UNK rate (0.00011%, 7 tokens) on characters absent from the training vocabulary.

Training both variants let us directly compare a byte-level and a character-level tokenizer under matched conditions (same script family, same general cleaning pipeline, similar fertility targets), rather than assuming one approach is preferable a priori. The trade-off is exactly what the results show: byte-level BPE guarantees zero UNK at the cost of an unreadable, Latin-byte-mapped vocabulary file, while character-level BPE keeps the vocabulary readable at the cost of a small but non-zero UNK rate.

4 Reproduction

Dataset collection scripts, preprocessing pipelines, and tokenizer training code for both languages are included in the repository, organized under separate `hindi/` and `nepali/` directories as specified in the assignment layout. Trained tokenizer and vocabulary files are included directly in the repository; raw and intermediate corpus files exceed the size suitable for git and are hosted externally.

- GitHub repository: `[TODO: add repo link]`
- Hindi corpus (Drive): `[TODO: add link]`
- Nepali corpus (Drive): `[TODO: add link]`
- Hindi tokenizer files: `[TODO: add link, if not in repo]`
- Nepali tokenizer files: `[TODO: add link, if not in repo]`

5 Summary

Both corpora meet or exceed the 500M-token target and the 20% manual-collection minimum, and both tokenizers were trained independently with no shared data or vocabulary between Model H and Model L. This satisfies the Phase 1 requirements ahead of the pretraining and evaluation work in Phase 2.